

阅读札记 · NETWORKING

Tailscale 《How NAT traversal works》中文导读

用我自己的话讲一遍 NAT 穿透这件事，并把每个概念落到 shuai-tunnel 的实际代码位置。

关于这份文档 这不是原文翻译。原文是 Tailscale 公司在他们博客上发表的技术长文，版权属于 Tailscale Inc. 我只是把那篇文章里的核心概念用自己的话重讲一遍，顺便把每个知识点对应到 shuai-tunnel 的代码位置，方便你把书本知识跟我们项目对上号。想看完整原文，去文末的链接。

一、为什么 P2P 这么难

假设你家里有台服务器，想让朋友家的电脑直接连过来。听起来很简单——TCP 三次握手嘛。但实际上你会发现根本连不上。原因是你们俩的电脑都躲在家用路由器（NAT）后面，谁都不知道对方的「真实地址」是什么，对方也根本看不见你这边的内网 IP。

IPv4 地址不够用，运营商不可能给每户每台设备分配一个公网 IP。现实里一个公网 IP 后面可能挂着几十台设备，靠路由器在出口做端口翻译来让所有设备共用一根带宽。这套东西就是 NAT。NAT 解决了「地址不够用」，却引入了「两边都在 NAT 后面就连不上」的难题，于是有了「穿透」这件事要做。

二、NAT 究竟在做什么

NAT 本质是带状态的防火墙加地址改写。当你内网 10.0.0.5:54321 主动给外网 1.1.1.1:80 发包，路由器做三件事：

- 把出包的源地址改写成路由器自己的公网 IP，源端口改写成一个新分配的端口（例如改成 203.0.113.7:33990）；
- 在内部的 NAT 表里登记一条映射条目，记下「(内网 10.0.0.5:54321) ↔ (公网 203.0.113.7:33990)」；
- 等外面回包到 203.0.113.7:33990，路由器查表，反向改写目的地址，把包送回 10.0.0.5:54321。

关键在于这条映射条目是路由器单方面建立的，外界既不知道它存在，也无法主动让它出现。如果外面有个对端想第一个发包给你，它根本不知道往哪个公网端口发；就算它瞎猜一个，路由器查表没找到映射，会把包丢掉。

三、经典 NAT 分类，以及为什么它已经过时

RFC 3489 给 NAT 分成了四类：Full Cone、Restricted Cone、Port Restricted Cone、Symmetric。这套分法在二十年前是行业通用语，但现在的研究和实务里已经不再死守，原因是它把两件实际上独立的事糅在了一个分类里：

类型	Mapping 行为	Filtering 行为
Full Cone NAT	同一内网端口 → 同一公网端口（与目标无关）	任何外网主机都能用这个映射回包
Restricted Cone NAT	同一内网端口 → 同一公网端口	只有发过包给目标 IP 时，目标 IP 才能回包
Port Restricted Cone NAT	同一内网端口 → 同一公网端口	只有发过包给目标 IP+Port 时，对方才能回包
Symmetric NAT	不同目标会产生不同公网端口	通常也是 Port Restricted 级别

前三种的映射规则其实是同一种（端口跟目标无关），只是回包过滤的严苛程度不同；Symmetric 才是映射本身就不一样。这两件事——「映射怎么分配」和「过滤多严」——在打洞策略上有完全不同的影响，把它们硬凑成 4 类反而让人糊涂。

四、现代视角：Mapping × Filtering 二维分类

现在更被广泛接受的是 RFC 4787 的二维分类。每一维都问一个独立的问题：

4.1 Mapping 维度（地址端口怎么分配）

类型	判定	对打洞的影响
Endpoint-Independent Mapping (EIM)	同一本机 socket 发往任何目标，公网映射端点都相同	对打洞友好。一次 STUN 探测，告诉所有对端这个端点即可
Address-Dependent Mapping (ADM)	同一本机 socket 发往同一公网 IP 时端点一致，不同 IP 不一致	需要每个对端分别探测，比较受限
Address+Port-Dependent Mapping (Symmetric)	目标 IP 或端口任一变化，公网端口就变	打洞通常失败。每次出包都拿到一个新端口，对端用旧端点回包根本不通

4.2 Filtering 维度（什么样的回包能进来）

类型	判定
Endpoint-Independent Filtering	只要映射存在，任何外网 IP 都能回包
Address-Dependent Filtering	必须是本机曾经发过包的目标 IP 才能回包
Address+Port-Dependent Filtering	必须是本机曾经发过包的 IP 和端口才能回包

关键洞察：打洞成功率主要看 Mapping 维度，不是 Filtering。Filtering 严不严没关系——只要 Mapping 是 EIM，两边同时给对方公网端点发包，Filtering 表里就会自动把对方加进来；打洞成功。反过来，Symmetric 这一类 Mapping，任何 Filtering 配合都救不回来，因为公网端口每次都在变，对端拿到的那个端口很快就失效了。

五、STUN：探出自己的公网身份

STUN (RFC 8489) 的功能可以一句话讲完：客户端用一个真实业务要用的 UDP socket 给 STUN 服务器发个空包，服务器回包告诉你「我看到你的源地址是 X.X.X.X:YYYYY」。客户端拿到这个 X.X.X.X:YYYYY

后，就知道自己对外的公网端点是什么，可以告诉其他对端「往这里发包能找到我」。

STUN 本身不打洞——它只是个「镜子」。打洞要靠两边同时发包，让各自的 NAT 都打开映射。STUN 让两边都能知道自己的公网端点是什么，然后通过某个外部信道（也就是控制面）交换这俩端点，再同时朝对方发包。

需要注意的是 STUN 在 Symmetric NAT 后面给出的结果对其他对端没用：你跟 STUN 服务器对话产生的那个公网端点，跟你跟对端 P2P 对话时产生的公网端点不是同一个。这是浏览器 NAT 检测页里判 Symmetric 的本质原理——我们让同一本机 socket 给两个 STUN 服务器发探测，看公网端点是否一致；不一致就是 Symmetric。

六、难啃的骨头：Symmetric NAT 与端口预测

Symmetric NAT 看似无解，但还有一招叫端口预测。如果 NAT 给新连接分配公网端口的策略是「顺序递增」或「在一个小池子里随机」，那么就有可能猜对。

更狠的一招借用生日悖论。假设 NAT 在 0~65535 端口里均匀随机给新映射，如果一方一次性朝对方喷 256 个不同源端口的包（每个都会在自家 NAT 上申请一个新端口），同时对方也朝你喷 256 个，那两边新建的映射条目里只要有一对「恰好对上」——你猜的端口被对方分配到，对方猜的端口被你分配到——打洞就成。 $256 \times 256 = 65536$ 种组合，已经覆盖整个端口空间，碰撞概率相当高。

代价是要烧大量端口、产生大量探测包，而且对一些做「端口分配随机化」的现代 NAT（特别是商用运营商机 NAT）效果会差很多。所以端口预测在 Tailscale 这类公司里只作为「在 relay 兜底之前再多挣扎一次」的策略，在生产里通常会快速放弃然后退到

relay。

七、ICE：双向打洞的协调框架

ICE（RFC 8445）是 WebRTC 里那套把「找端点 + 试打洞」系统化的框架。ICE 的核心思路有三步：

- 收集候选端点（Candidate Gathering）：每端尽可能列出自己的所有可触达地址——本机网卡 IP（host candidate）、STUN 给出的公网映射（srflx candidate）、TURN relay 分配的中继端点（relay candidate）。
- 通过信令交换候选（Signaling）：两边把各自的候选列表通过某个外部通道（控制面服务器、WebSocket、对端控制器）发给对方。ICE 协议本身不规定信令怎么实现，因为打洞之前两边还没法直接通信。
- 并发探测（Connectivity Checks）：两边对所有（本地候选 × 对端候选）组合发送 STUN binding 探测，按优先级排序。第一个能双向通的组合就被选为最终路径。

ICE 的好处是不要赌：不预先假定哪条路一定通，把所有候选都试一遍并发跑，让网络自己决定。失败的探测包成本很低，成功的那条路立刻能用。

八、UPnP / NAT-PMP / PCP：让 NAT 主动配合

上面所有招都是「猜」或「试」，因为客户端跟自家 NAT 没有协商的语言。但其实有几种协议是专门让客户端跟自家路由器说「给我开个洞」的：

协议	组织 / 年代	状态
UPnP IGD	UPnP Forum / 2001	消费级路由器最广泛支持，但默认很多都关闭
NAT-PMP	Apple / 2008	苹果生态、部分开源路由（OpenWrt）支持
PCP (Port Control Protocol)	IETF RFC 6887 / 2013	NAT-PMP 的后继者，更适合 IPv6 和 CGNAT 场景

这些协议如果可用，效果好得多——客户端不用 STUN，不用猜端口，直接告诉路由器「我要把公网某端口转发到我内网某端口」，路由器照办。实际里能不能用要看用户路由器有没有开。Tailscale 的客户端会同时尝试这三种协议，如果都失败再退回到 STUN + 打洞那一套。

九、Relay：最后的保险绳

无论你多聪明，总有些网络是怎么打都打不通的：某些运营商机房 NAT（CGNAT）严格 Symmetric，某些企业网关只放行 HTTPS 出站，某些校园网把 UDP 全锁了。这时候唯一能保证可用的兜底就是 relay（中继）：客户端 A 把流量发到一台公网服务器，由服务器转发给客户端 B。

标准方案是 TURN (RFC 8656)，它是 STUN 协议的超集，客户端通过 TURN 服务器「分配一个 relay 端口」，所有发到这个端口的流量都会被服务器转发给客户端。TURN 必须做鉴权（否则就是免费代理给全互联网用），所以一般跟控制面绑定，由控制面下发短期凭证。

Tailscale 自己实现了一套叫 DERP (Designated Encrypted Relay for Packets) 的 relay 网络，基于 HTTPS (TCP/443) 而不是 UDP，这样能穿过几乎所有企业网关。代价是 TCP 引入额外的延迟和拥塞控制重传，但能让「哪怕只有 HTTPS 能出」的极端网络也能跑起来。

设计要点：relay 不是最优选择，是兜底。客户端必须持续尝试找直连路径，一旦发现就切走。否则所有流量都吃服务器带宽，运维成本会失控。

十、一句话总结

NAT 穿透 = 在控制面协调下，两端并发尝试所有可能的连接路径，按优先级选出第一条能用的，找不到就退到 relay。这套思路是 ICE 框架的核心，是 WebRTC、Tailscale、shuai-tunnel 这类系统的共同地基。区别只在每家挑了哪些候选类型、用什么信令、以及 relay 走 UDP 还是 TCP。

十一、对应到 shuai-tunnel 的实现

把上面这些概念落到我们项目的代码里：

概念	shuai-tunnel 里的位置
控制面信令	PEER_CONTROL 命令通过 7010 Netty 控制连接收发；PeerMeshService 转发信令
身份与鉴权	客户端 X25519 静态密钥对，公钥登记在服务端；会话 token 由控制面短期签发
STUN 探测	客户端 STUN/TURN-lite 主端口 3478/udp，辅助端口 3479/udp 用来判 Mapping 类型
Candidate 类型	host (本机 IP) / srflx (STUN 看到的公网端点) / relay (TURN-lite 中继端点)
NAT 类型分类	客户端把探测结果归为 NO_NAT / PORT_PRESERVED / CONE_LIKE / SYMMETRIC / NAT 五类 (见 lib/nat.ts)
数据面加密	Peer 之间用对称密钥加密 UDP frame，复用 X25519 + ChaCha20-Poly1305 类似 Noise
Relay 兜底	服务端自带 TURN-lite，分配 relay 端点；客户端直连失败后自动切到 relay
路径切换	数据面持续探测直连可用性，可用就立刻切回 direct，避免长期吃服务端带宽

当前简化：shuai-tunnel 没有实现 UPnP / NAT-PMP / PCP，也没有 Tailscale 那种基于 TCP/443 的 relay (我们的 TURN-lite 是 UDP)。这是有意为之，第一版先把控制面和加密数据面跑通，极端网络情况 (只放 HTTPS

出站的企业网) 目前会回退到 server 中继的 HTTP 直转通道, 不走 peer mesh 数据面。

十二、继续深入

Tailscale: How NAT traversal works

<https://tailscale.com/blog/how-nat-traversal-works>

本文的灵感来源。讲得详细、风格友好, 强烈推荐去读原文。

RFC 8489 – Session Traversal Utilities for NAT (STUN)

<https://www.rfc-editor.org/rfc/rfc8489>

STUN 主协议文档。看字段编号和帧结构要查这个。

RFC 8445 – Interactive Connectivity Establishment (ICE)

<https://www.rfc-editor.org/rfc/rfc8445>

ICE 框架。Candidate 收集、优先级算法、双向探测都在这里。

RFC 4787 – NAT Behavioral Requirements for UDP

<https://www.rfc-editor.org/rfc/rfc4787>

Mapping × Filtering 二维分类的原始定义。

RFC 8656 – TURN: Traversal Using Relays around NAT

<https://www.rfc-editor.org/rfc/rfc8656>

TURN 中继协议。理解 relay 兜底机制看这个。

RFC 6887 – Port Control Protocol (PCP)

<https://www.rfc-editor.org/rfc/rfc6887>

让客户端跟 NAT 协商显式端口映射。

本文为基于 Tailscale 同名博客的中文导读, 由 shuai-tunnel 项目整理。原文版权归 Tailscale Inc., 本文不复述原文内容, 仅讨论公共领域的网络协议知识并联系本项目代码。